

Full Stack Development with MERN

Project Documentation

Script-Controlled ACL - Restrict Record Access Based on Field Value

1. Introduction

Project Title: Script-Controlled ACL - Restrict Record Access Based on Field Value

Team Members: [Add team member names and roles]

This project demonstrates a field-based Access Control List (ACL) mechanism in a MERN application. Access to individual records is determined dynamically by a server-side authorization script that evaluates a selected field value before allowing a user to view or operate on the record.

2. Project Overview

Purpose:

The purpose of the project is to implement record-level authorization where access is controlled by the value of a field in the record. For example, a record may contain an 'accessLevel' field such as 'public', 'team', or 'private'. The backend evaluates the authenticated user's role and the record's field value before granting access.

Features:

User authentication and role-based authorization.

Field-value-based record access control.

Server-side ACL evaluation before protected operations.

Protected API endpoints for viewing and modifying records.

MongoDB storage for users and controlled records.

Clear authorization responses for denied requests.

Extensible authorization logic that can support additional field values and rules.

3. Architecture

Frontend:

The React frontend provides login, record listing, record details, and access-status interfaces. It communicates with the backend through REST APIs and does not make the final authorization decision.

Backend:

Node.js with Express.js exposes authentication and record-management APIs. Middleware authenticates the user, loads the requested record, and invokes the ACL script before protected operations are completed.

Database:

MongoDB stores user information and records. A controlled record can include fields such as title, description, ownerId, accessLevel, and timestamps. The accessLevel value is evaluated by the ACL logic.

Authorization Flow:

User signs in and receives an authentication token.

React sends the token with a protected API request.

Express middleware validates the token and identifies the user.

The backend retrieves the requested record from MongoDB.

The Script-Controlled ACL evaluates the user's role/identity and the record's field value.

If the rule returns allow, the requested operation continues; otherwise, the API returns an authorization error.

4. Setup Instructions

Prerequisites:

Node.js and npm installed.

MongoDB installed locally or a MongoDB Atlas connection.

A modern web browser.

Basic knowledge of React, Express.js, and MongoDB.

Installation:

Create or clone the project repository.

Install frontend dependencies using npm install inside the client directory.

Install backend dependencies using npm install inside the server directory.

Create the backend environment file and configure the MongoDB connection string and authentication secret.

Start MongoDB if using a local database.

Run the frontend and backend using the commands provided in Section 6.

Example environment variables:

MONGO_URI=<your-mongodb-connection-string>JWT_SECRET=<your-authentication-secret>PORT=5000

5. Folder Structure

Client:

```
client/├─ src/|   └─ components/|   └─ pages/|   └─ services/|   └─ App.jsx|   └─ main.jsx└─ package.json
```

Server:

```
server/├─ controllers/├─ middleware/|   └─ authMiddleware.js|   └─ aclMiddleware.js├─ models/|   └─ User.js|   └─ Record.js├─ routes/├─ scripts/|   └─ accessControl.js├─ config/├─ server.js└─ package.json
```

The `aclMiddleware.js` component connects the authenticated request with the ACL script. The `accessControl.js` script contains the field-value-based authorization rules.

6. Running the Application

Frontend:

```
cd clientnpm installnpm start
```

Backend:

```
cd servernpm installnpm start
```

After both servers are running, open the frontend URL shown by the React development server in a browser. The frontend should communicate with the Express backend through the configured API base URL.

7. API Documentation

The following endpoints represent the core API design for the project.

Method	Endpoint	Purpose	Authorization
POST	/api/auth/login	Authenticate a user and return a token.	Public
GET	/api/records	List records available to the authenticated user.	Authenticated
GET	/api/records/:id	Retrieve one record after ACL evaluation.	Authenticated + ACL
POST	/api/records	Create a new controlled record.	Authenticated
PUT	/api/records/:id	Update a record after ACL evaluation.	Authenticated + ACL
DELETE	/api/records/:id	Delete a record after ACL evaluation.	Authenticated + ACL

Example ACL decision:

```
// Example conceptual ruleif (record.accessLevel === "public") {    return true;}if (record.accessLevel === "team" && user.role === "member") {    return true;}if (record.accessLevel === "private" && record.ownerId === user.id) {    return true;}return false;
```

Example successful response:

```
{  "success": true,  "data": {    "id": "record_id",    "title": "Project Record",    "accessLevel": "team"  }}
```

Example denied response:

```
{  "success": false,  "message": "Access denied for this record"}
```

8. Authentication

Authentication identifies the user making the request, while authorization determines whether that user may access a particular record. A token-based approach such as JWT can be used for authentication.

The user submits valid credentials to the login endpoint.

The server verifies the credentials and issues an authentication token.

The client sends the token with subsequent protected requests.

Authentication middleware verifies the token and attaches the user information to the request.

ACL middleware evaluates the authenticated user against the target record's field value.

Unauthorized users receive an appropriate HTTP authorization error instead of the protected record.

Important security principle: the frontend may hide records from the interface for convenience, but the backend ACL must always enforce the actual access restriction.

9. User Interface

The React interface can include the following screens:

Login page for user authentication.

Dashboard displaying records the current user is permitted to access.

Record details page showing permitted record information.

Create/Edit record form containing the accessLevel field.

Access-denied message when the server rejects a protected request.

Screenshots should be added here after the application is implemented and tested.

10. Testing

Testing should verify both successful access and denied access. API testing tools such as Postman can be used alongside frontend testing.

Test access to a public record with an authenticated user.

Test team-level records with an authorized and unauthorized user.

Test private records using the owner and a different user.

Test access with an invalid or expired token.

Test attempts to modify records without sufficient permission.

Test records containing unsupported or unexpected accessLevel values.

Verify that denied requests do not expose protected record data.

11. Screenshots or Demo

Add screenshots of the login page, record dashboard, record details, ACL-controlled access result, and access-denied screen. A project repository or deployed demo link may also be added here.

Demo/Repository Link: [Add link here]

12. Known Issues

The exact access rules depend on the application's configured ACL script.

If authorization logic is changed without corresponding tests, unexpected access behavior may occur.

The project requires secure management of JWT secrets and database credentials.

Screenshots and demo links must be added after the application is completed.

13. Future Enhancements

Create an admin interface for managing ACL rules without changing source code.

Support multiple field-based conditions using AND/OR rule combinations.

Add audit logs for allowed and denied access attempts.

Support more granular permissions such as read, create, update, and delete.

Add automated unit and integration tests for ACL rules.

Introduce role and permission management from the frontend.

Add configurable policies for different record types.